OpenMP Tasks: `task` and `taskwait`

Lesson 12: recursive Fibonacci and task granularity

Gustavo Lozano

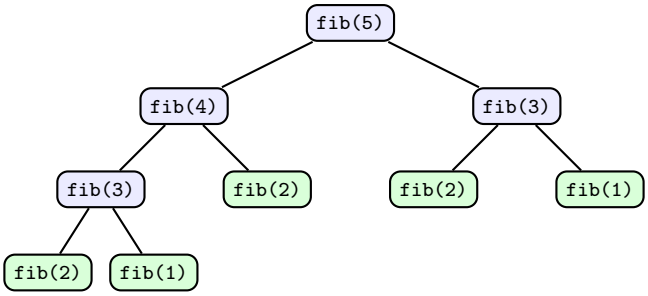
November 2026

Source: van der Pas, Stotzer & Terboven [1], chapter on tasking. Tasks were added in OpenMP 3.0 (2008), after the Chapman book.

1 The idea

Everything so far divided a *loop* (`for`) or a fixed list of blocks (`sections`). Much work has neither shape: recursion, tree and graph traversals, linked lists, divide-and-conquer algorithms. The number and size of the pieces is only known while the program runs.

A **task** is a piece of work, code plus its data, that a thread puts into a *queue*. Any thread of the team may take it from the queue and execute it, now or later. One thread can create tasks, and tasks can create more tasks; the idle threads keep taking them. `taskwait` makes a task wait until the tasks *it* created (its children) have finished.



`fib(5)` with cutoff 3. Blue: creates two tasks and waits for them (`taskwait`); green: below the cutoff, computed directly without tasks.

2 The pattern

```
1  #pragma omp parallel          /* create the team */
2  {
3      #pragma omp single        /* ONE thread starts the work ...            */
4      r = fib(n, cutoff);       /* ... the others execute the tasks it makes */
5  }                             /* barrier: all tasks are finished here      */
```

Inside `fib`:

```
1  #pragma omp task shared(a)    /* put "a = fib(n-1)" in the queue */
2  a = fib(n - 1, cutoff);
3  #pragma omp task shared(b)    /* put "b = fib(n-2)" in the queue */
4  b = fib(n - 2, cutoff);
5  #pragma omp taskwait          /* wait for my two children        */
6  return a + b;
```

Data in tasks. A task may run after the function that created it has moved on, so OpenMP gives it a **copy** of the creating function's local variables: they are `firstprivate` by default (Lesson 7). Without `shared(a)`, the task would write its result into its own copy of `a`, which is thrown away, and `fib` would return a wrong value (removing both clauses printed `fib(30) = 0` in a test). `shared(a)` is safe here because `taskwait` guarantees the task has finished before `a` is read, and `a` still exists at that point.

Granularity. Creating, queuing and scheduling a task costs time. If a task does less work than that, tasks make the program *slower*. The `cutoff` stops creating tasks for small sub-problems and computes them directly: tasks are created only for `n >= cutoff`.

3 The code

```
/*
 * fib_tasks.c -- OpenMP Lesson 12: tasks and taskwait.
 * Source: van der Pas, Stotzer & Terboven (2017), "Using OpenMP -- The Next
 *         Step", chapter on tasking.
 *
 * Fibonacci numbers by the (deliberately slow) recursion
 *     fib(n) = fib(n-1) + fib(n-2),  fib(0) = 0, fib(1) = 1.
 * The recursion is a tree of calls, not a loop: "omp for" cannot split it,
 * but tasks can.
 *
 * Compile:
 *     gcc -Wall -O2 -fopenmp -o fib_tasks fib_tasks.c
 * Run:   ./fib_tasks n cutoff
 *     OMP_NUM_THREADS=4 ./fib_tasks 40 20
 */

#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

/* plain recursion, no tasks */
static long fib_serial(int n)
{
    return n < 2 ? n : fib_serial(n - 1) + fib_serial(n - 2);
}

/* Each call creates two tasks, one per sub-problem, and waits for both.
   Below the cutoff the work is too small to be worth a task: compute it
   directly.                                                              */
static long fib(int n, int cutoff)
{
    if (n < cutoff)
        return fib_serial(n);

    long a, b;

    /* A task is a piece of work put in a queue. ANY thread of the team may
       execute it, now or later. Variables of this function are
       firstprivate in a task by default (a copy!), so a and b must be
       shared, or the results would be written into the task's copies.    */
    #pragma omp task shared(a)
    a = fib(n - 1, cutoff);

```

```
44      #pragma omp task shared(b)
45      b = fib(n - 2, cutoff);
46
47      /* wait until the two child tasks above have finished */
48      #pragma omp taskwait
49      return a + b;
50  }
51
52  int main(int argc, char **argv)
53  {
54      int n      = (argc > 1) ? atoi(argv[1]) : 40;
55      int cutoff = (argc > 2) ? atoi(argv[2]) : 20;
56      long r;
57
58      double t = omp_get_wtime();
59      r = fib_serial(n);
60      double t_serial = omp_get_wtime() - t;
61      printf("serial : fib(%d) = %ld   %.3f s\n", n, r, t_serial);
62
63      t = omp_get_wtime();
64      #pragma omp parallel        /* create the team of threads ...           */
65      {
66          #pragma omp single      /* ... but only ONE thread starts the recursion;
67                                     the others pick up the tasks it creates   */
68          r = fib(n, cutoff);
69      }                           /* all tasks are finished at the barrier here */
70      double t_tasks = omp_get_wtime() - t;
71
72      printf("tasks  : fib(%d) = %ld   %.3f s   (threads = %d, cutoff = %d, speed-up =
        %.2f)\n",
73             n, r, t_tasks, omp_get_max_threads(), cutoff, t_serial / t_tasks);
74      return 0;
75  }
```

Listing 1: **`fib_tasks.c`** – recursive Fibonacci with tasks and a cutoff.

```
gcc -Wall -O2 -fopenmp -o fib_tasks fib_tasks.c
OMP_NUM_THREADS=4 ./fib_tasks 42 25
```

Output on a 2-core machine (your times will differ):

```
serial : fib(42) = 267914296   0.545 s
tasks  : fib(42) = 267914296   0.339 s   (threads = 2, cutoff = 25, speed-up = 1.61)
```

Key ideas

- Tasks parallelize work that is not a loop: one thread creates them, every thread of the team executes them.
- Use `parallel` + `single` to start: one creator, a full team of workers.
- `taskwait` waits for the *children* of the current task (not for grandchildren; each child waits for its own).
- Local variables are `firstprivate` in a task: results must go to a `shared` variable that lives until the `taskwait`.
- A task must be worth its overhead: use a cutoff.

4 The experiment

Experiment

Question: how large should a task be?

1. Run `fib(42)` with all your cores and several cutoffs:

```
export OMP_NUM_THREADS=$(nproc)
for c in 10 15 20 25 30 35 42; do
    ./fib_tasks 42 $c | tail -1
done
```

2. Note the speed-up for each cutoff. The number of tasks each cutoff creates is:

cutoff	10	15	20	25	30	35	42
tasks created	18 454 928	1 664 078	150 048	13 528	1 218	108	2

What to observe and answer.

1. With cutoff 10 the parallel version is much *slower* than the serial one. Why? Estimate the cost of one task from your times.
2. With cutoff 42 only 2 tasks are created, `fib(41)` and `fib(40)`. What is the best speed-up possible, whatever the number of threads?
3. Which cutoff is best on your machine, and what does it say about the right number of tasks compared with the number of threads?

Answers (read after trying). (1) Almost every task is tiny: `fib(10)` takes well under a microsecond, about the same as creating and scheduling a task. With $1.8 \cdot 10^7$ tasks the overhead dominates. On the 2-core test machine cutoff 10 took 11.5 s against 0.55 s serial: roughly $(2 \times 11.5 - 0.55)\,\text{s}/1.8 \cdot 10^7 \approx 1.2\,\mu\text{s}$ of thread time per task. (2) The two tasks are unequal: `fib(41)` is about 62% of the work (fib(41)/fib(42) $\approx 1/\varphi \approx 0.618$). The run cannot be shorter than that largest task, so the speed-up is at most $1/0.618 \approx 1.6$, even with 64 threads. Too few tasks means poor load balance. (3) Usually a cutoff around 25–35: a few hundred to a few thousand tasks. That is many more tasks than threads, so idle threads always find work (balance), yet each task is large enough to hide its overhead. The rule: many more tasks than threads, each much more expensive than creating it.

Next lesson. `taskloop` and task dependencies (`depend`): ordering tasks by the data they read and write, without barriers.

References

[1] R. van der Pas, E. Stotzer, and C. Terboven. *Using OpenMP — The Next Step: Affinity, Accelerators, Tasking, and SIMD*. MIT Press, Cambridge, MA, 2017.